



Lightweight Multitenancy at the Network's Extreme Edge

Peng Liu, Lance Hartung, and Suman Banerjee, University of Wisconsin–Madison

Many Internet services require lower latency, greater responsiveness, and more efficient use of network bandwidth. As a result, researchers have espoused the need for edge computing, which distributes services closer to user devices. ParaDrop is an edge-computing platform that allows users and developers to deploy diverse services in homes and enterprises.

Cloud computing platforms, such as Amazon Web Services, Microsoft Azure, and Google Cloud Platform, have become popular for providing ubiquitous access to services across a range of user devices. Cloud platform providers (Amazon, Microsoft, and the like—known as infrastructure providers) take advantage of economies of scale by managing and operating resources in a centralized manner.

Cloud computing provides reliable, scalable, and elastic resources, which is motivating more and more developers to deploy their services and applications in the cloud. Netflix and Dropbox are examples of popular cloud-based services. Users can access high-performance computing and large storage resources anywhere with Internet access at any time. Cloud-based services' centrally managed resources and applications help users by eliminating requirements to install and configure software locally on mobile devices with relatively low-performance computing resources and small

storage space. Google Docs and Adobe Creative Cloud are examples of cloud services that provide traditional office tools in a convenient manner.

Cloud services require developers to host services, applications, and data on off-site datacenters, meaning the computing and storage resources are spatially distant from users' devices. However, a growing number of high-quality services want computational tasks to be located nearby. These services require lower latency, greater responsiveness, a better user experience, and more efficient use of network bandwidth. As a result, over the last decade a number of research efforts have espoused the need and benefits of creating edge-computing services that distribute computational functions closer to the client devices, such as cyber foraging,¹ cloudlets,² and more recently, fog computing³ and mobile edge computing.⁴ For information about previous work related to edge computing, see the sidebar.

RELATED WORK ON EDGE COMPUTING

We present an edge-computing platform, ParaDrop, implemented on Wi-Fi access points (APs) or other wireless gateways such as set-top boxes. ParaDrop allows third-party service developers to bring computational functions into homes and enterprises. The Wi-Fi AP is ubiquitous, always “on” and available, and sits directly in the data path between the Internet and user devices. For these reasons, we selected the Wi-Fi AP for ParaDrop as the edge-computing node.

PARADROP

ParaDrop uses a lightweight virtualization framework through which third-party developers can create, deploy, and revoke their services in different APs. These services are housed in isolated containers (called “chutes,” short for “parachutes”—as in parachuting a service on demand) that allow them to retain user state and move with users as they change their points of attachment. Because Wi-Fi APs are likely to be resource-limited for many different types of applications, such as video analysis and data caching, ParaDrop allows for tight resource control through a managed policy design. Unlike cloud infrastructure where providers own all resources, ParaDrop encourages users to contribute their resources to the platform. This different ownership model leads to more complex permission management and resource policies than in cloud computing.

The ParaDrop platform has three key components: a virtualization substrate in the Wi-Fi APs that hosts third-party computations in chutes; a cloud back end (controller) through which all of the third-party services are dynamically installed, instantiated, and revoked; and a developer API through which developers can

Many researchers have explored the advantages of edge computing and proposed different approaches to using them. Rajesh Balan and his colleagues presented cyber foraging, a mechanism to augment the computational and storage capabilities of mobile devices.¹ Tolga Soyata and his colleagues developed MOCHA, a mobile/cloudlet/cloud architecture that partitions tasks from mobile devices to the cloud and distributes compute load among cloud servers (cloudlets) to minimize the response time.² Kiyong Ha and his colleagues described the architecture and prototype implementation of an assistive system based on Google Glass devices that utilizes edge computing.³ Tan Zhang and his colleagues built Vigil, a real-time distributed wireless surveillance system that leverages edge computing to support real-time tracking and surveillance in different scenarios.⁴

Mahadev Satyanarayanan and his colleagues proposed Cloudlet, a trusted, resource-rich computer or cluster of computers that has a strong connecting to the Internet and is available for use by nearby mobile devices.⁵ Flavio Bonomi and his colleagues discussed fog computing, which brings data processing, networking, storage, and analytics closer to mobile devices—they argued that the characteristics of fog computing make it the appropriate platform for many critical Internet of Things services and applications.⁶ Finally, industry is currently standardizing mobile edge computing, which provides computation and storage resources within the radio access network (RAN) in close proximity to mobile subscribers to reduce latency, ensure highly efficient network operation, and improve user experience.⁷

References

1. R. Balan et al., “The Case for Cyber Foraging,” *Proc. 10th ACM SIGOPS European Workshop*, 2002, pp. 87–92.
2. T. Soyata et al., “Cloud-Vision: Real-time Face Recognition Using a Mobile-Cloudlet-Cloud Acceleration Architecture,” *IEEE Symp. Computers and Communications (ISCC 12)*, 2012; www.tolgasoyata.com/mocha/pdfs/soyata.iscc12.pdf.
3. K. Ha et al., “Towards Wearable Cognitive Assistance,” *Proc. 12th Ann. Int’l Conf. Mobile Systems, Applications, and Services (MobiSys 14)*, 2014, pp. 68–81.
4. T. Zhang et al., “The Design and Implementation of a Wireless Video Surveillance System,” *Proc. 21st Ann. Int’l Conf. Mobile Computing and Networking (MobiCom 15)*, 2015, pp. 426–438.
5. M. Satyanarayanan et al., “The Case for VM-based Cloudlets in Mobile Computing,” *IEEE Pervasive Computing*, vol. 8, no. 4, 2009, pp. 14–23.
6. F. Bonomi et al., “Fog Computing and its Role in the Internet of Things,” *Proc. 1st Edition MCC Workshop Mobile Cloud Computing*, 2012, pp. 13–16.
7. Y.C. Hu et al., *Mobile Edge Computing: A Key Technology Towards 5G*, white paper, ETSI, 2015; www.etsi.org/images/files/ETSIWhitePapers/etsi_wp11_mec_a_key_technology_towards_5g.pdf.

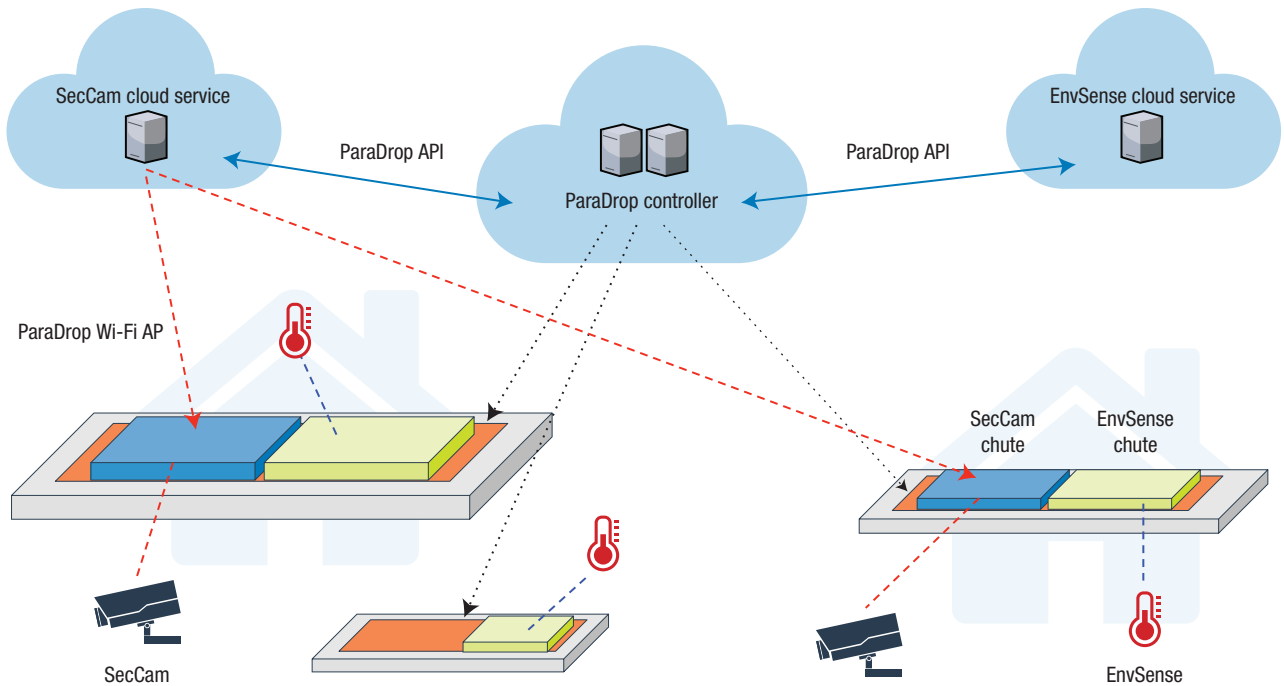


FIGURE 1. ParaDrop overview. The system is made up of three components: ParaDrop access points (APs), the ParaDrop controller, and the ParaDrop API. Developers deploy services in containers to the APs through the ParaDrop controller by calling the API. Two different applications are illustrated here: SecCam, a wireless security camera service that collects video from an in-range camera and performs local processing to detect motion; and EnvSense, a system deployed in homes and buildings to monitor temperature and humidity with sensors.

manage the resources of the platform and monitor the running status of APs and chutes.

Figure 1 gives an overview of ParaDrop's architecture and shows two example applications. The first is SecCam, a security camera application that connects with wireless cameras in range to detect motion. The second, EnvSense, connects to environment sensors in the home—such as temperature and humidity—to determine the position of the actuator-controlled heating or cooling vents. With ParaDrop, services can be dynamically delivered from the cloud to the edge node through the ParaDrop API.

In addition to low latency and reduced network traffic, ParaDrop also has unique privacy advantages. For example, a third-party developer can create a ParaDrop service that allows sensitive user data such as a video feed to reside locally in the ParaDrop AP, instead of sending a continuous feed over the open Internet.

DESIGNING AN EDGE-COMPUTING PLATFORM

Managing resources in a centralized manner through virtualization techniques is the key to the success and advantages of cloud computing. The underlying resource management and operations are transparent to developers and users. With edge computing, we want to provide the same transparency to both developers and users. Installing a server and related software in a large building without centralized management is not the version of cloud computing most people are familiar with. Similarly, effective edge computing involves more than just deploying resources at the edge of the Internet. With ParaDrop, we introduce a complete framework for management and operation that connects edge nodes and exposes their resources. The framework forms a platform including a pool of resources distributed in close proximity to mobile devices and users. The platform provides an API for both

developers and users to manage services and resources in a secure and transparent way.

To design an effective and efficient edge-computing platform, the following questions need to be answered:

- ▶ Where in the edge of the network should resources be deployed?
- ▶ How do we effectively manage and provision the resources in the edge nodes?
- ▶ How do we define the API that exposes the platform's full capabilities?

We address these questions through our experience in building ParaDrop and relate our answers to design decisions that are particular to ParaDrop.

Deploying resources at the edge

Let us review the path that a packet takes from a mobile device to a server in the cloud. As shown in Figure 2, a

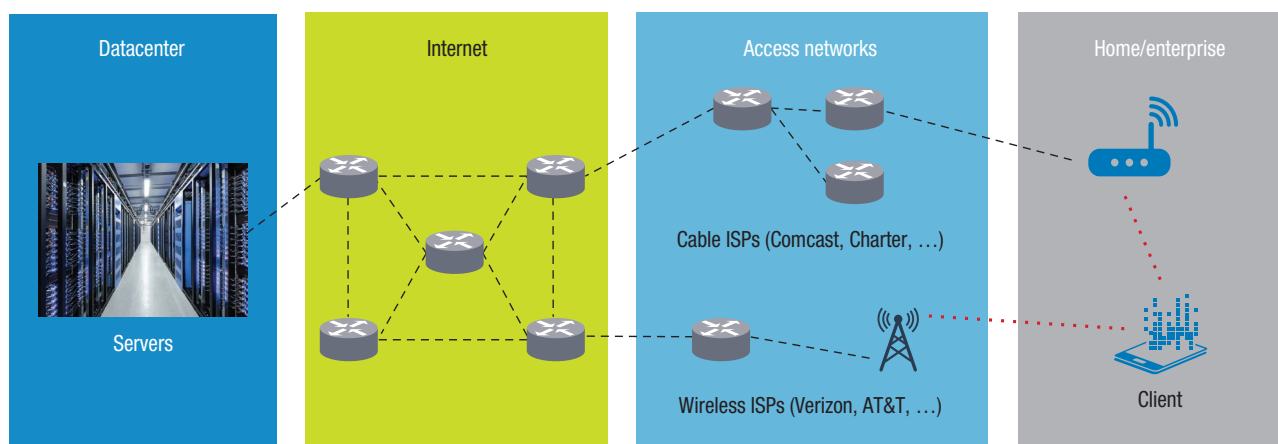


FIGURE 2. Overview of a packet's path from a mobile device to a cloud server. The packet goes through the home/enterprise network, the access network, the Internet, and eventually the datacenter network to reach the cloud server in the datacenter. ISP: Internet service provider.

packet from the mobile device goes through the home/enterprise network, the access network, the Internet, and eventually the datacenter network to reach the server in the datacenter. If the mobile device uses a cellular network instead of Wi-Fi, the home/enterprise network will be skipped. To eliminate the unpredictable latency due to congestion or bufferbloat⁵ in the Internet and datacenter network, as well as reduce network traffic, we can deploy resources in either the access network or home/enterprise network. Mobile edge computing⁴ chooses the first option, whereas ParaDrop chooses the latter—the extreme network edge.

Two decades ago, a desktop computer was the only reliable computing platform in the home where third-party applications could reliably and persistently run. However, diverse mobile devices, such as smartphones and tablets, have deprecated the desktop computer. With user devices going mobile, there is one remaining device that provides the capabilities developers require for their services as well as the proximity expected from an edge-computing framework. The gateway—which could be a home Wi-Fi AP or a cable set-top box provided by a network operator—is a platform that is continuously on and is a primary entry point into the end-user premises for such third-party services. Therefore, we chose to build the edge nodes with Wi-Fi APs.

Figure 3 shows an edge node of the ParaDrop platform—a ParaDrop Wi-Fi AP, which shares its resources with two wireless devices, SecCam and EnvSense.

Managing massive distributed resources

As mentioned earlier, cloud infrastructure providers have ownership of resources. Developers rent resources from infrastructure providers to deploy services. However, in an edge-computing platform, users can contribute their own resources to the edge-computing platform. As a result, an edge-computing platform has more diverse and distributed resources to manage than cloud computing. Edge computing can still use virtualization techniques to manage resources, but the management needs to be implemented in a highly distributed manner. Also, the number of edge nodes might be very large, so the scalability of the management framework is an important consideration in system design. We wanted to design a specialized management framework for edge-computing platforms instead of using current management frameworks for cloud computing.

To motivate users to delegate part of the resources in their Wi-Fi APs to the platform, the platform needs to provide real benefits such as improved user experience. Internet service providers (ISPs) can also contribute to the

platform by authorizing part of the resources (home gateways) to developers to deploy their services. ISPs can be motivated by service charges or reducing traffic entering their networks.

Exposing edge-computing capabilities through an API

In cloud computing, resource deployment and operations are transparent to developers and users. Providing similar transparency to developers and users in an edge-computing platform is critical for the platform's adoption.

We needed to develop an interface that both users and ISPs can use to provision resources on the platform. Built on top of the resources distributed at the Internet's edge, the platform provides an abstract view to service developers. In the abstract view, the resources at the Internet's edge are the same as those in the cloud (datacenters). Developers can use the platform API to deploy services from the cloud to the edge transparently. If developers design their services using micro-services architecture, this kind of partitioning and deployment will be a relatively easy task.

To support traditional smart appliances, an edge-computing platform also needs to support deploying services into the edge node manually by advanced users if they choose to do so. For example, an advanced user might want to deploy a dashboard application to manage all Internet

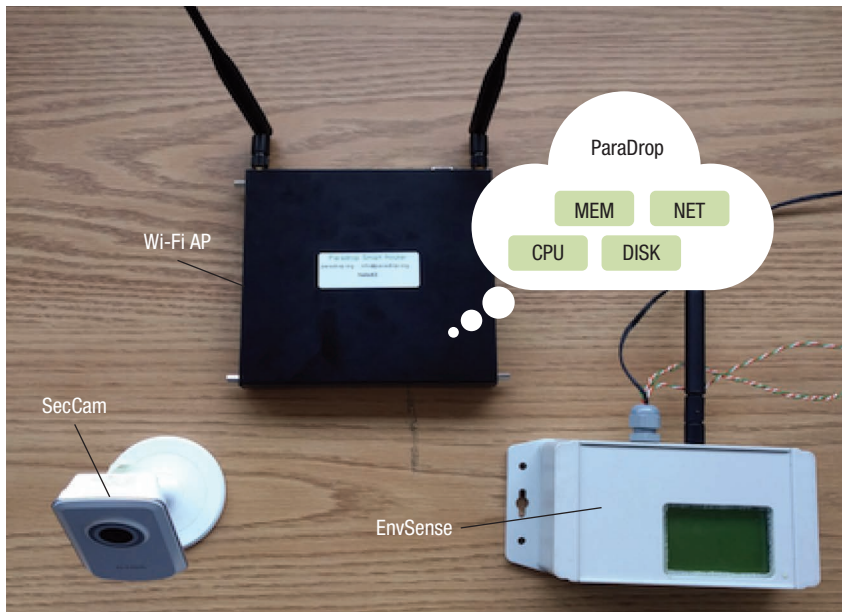


FIGURE 3. The fully implemented ParaDrop AP, which shares its resources with two wireless devices—SecCam and EnvSense.

of Things (IoT) sensors at home. This support also enables some standalone edge-computing applications that do not rely on a back end in the cloud, such as facial detection or other privacy-sensitive applications.

From the implementation perspective, the edge-computing API can be divided into two parts: the API implemented by the controller in the cloud and the API implemented by the edge nodes. The first exposes the platform's high-level resource and permissions management to developers, whereas the second is primarily used to expose an individual edge node's capabilities to the services running on the edge nodes.

DESIGNING PARADROP

Here, we discuss the challenges of implementing an edge-computing platform and how we overcome them in ParaDrop.

Highly efficient virtualization in edge nodes

The current generation of ParaDrop AP is based on the single board computer PEngines AP01. We choose Ubuntu Core—an embedded OS designed for IoT devices—as the OS

for the edge nodes. Ubuntu Core has a smaller footprint than Ubuntu's desktop and server versions, and it includes a package-management system that supports transactional updates. Unlike servers in datacenters, it is difficult to repair the hardware and software on an AP after deploying ParaDrop APs at the network edge (a user's home or office). Ubuntu Core provides secure and transactional update capability, and supports image backup and roll-back features so that the software running on APs can be managed easily and flexibly.

Ubuntu Core has a similar runtime environment as other Ubuntu versions—the similarity of the edge nodes to cloud servers in both hardware and software simplifies the task of deploying services from the cloud to the edge.

Wi-Fi APs have relatively limited resources compared to servers in datacenters for cloud computing. Therefore, efficiency was an important consideration in choosing the virtualization technique for ParaDrop. ParaDrop implements virtualization with Docker,⁶ an application layer virtualization solution based on Linux containers.⁷ This technique allows

ParaDrop to effectively use the limited resources available at the network edge. It provides higher performance virtualization than hypervisor-based virtual machines (VMs), especially in I/O speeds,⁸ which is important for latency-sensitive applications. Moreover, a container is more lightweight and faster than a VM because it does not run a full OS on the virtual hardware.

We needed to be very careful about resource policies because the resources on the edge nodes are shared between services and with the resource owners (the owner of the Wi-Fi AP). The first objective is to guarantee the services running on the edge node will never impact the users' tasks—in other words, the policy enforcement module in the AP needs to reserve resources for the basic function of the AP (Wi-Fi network access). In addition, owners (users or ISPs) of the Wi-Fi APs grant permissions to use some resources on the APs to ParaDrop, which in turn grants the permissions and quotas to the services deployed in the APs. Different services running on one AP are isolated from each other.

Scalable orchestration framework

Generally speaking, the ParaDrop platform needs to be capable of managing a larger number of nodes than a cloud-computing platform. To make matters worse, the edge nodes are distributed, and in many cases, the controller does not have direct access to the edge nodes due to firewalls. Therefore, it is difficult for developers to manage and debug services running on APs. Creating a virtual private network (VPN) is one approach to overcome this challenge; however, with a VPN, a centralized controller needs to maintain connections to all

APs. Developers also need to configure the VPN on their machines to access the APs. To simplify the controller implementation and developers' work, we used web protocols—HTTP and WebSocket—to enable bidirectional communication channels between the ParaDrop controller and the APs. With WebSocket, we can transmit time-sensitive messages among the controller, developer tools, and APs with low latency. As the number of managed APs and developers increases, the controller can be scaled up by using readily available tools and techniques for web applications such as replication and load balancing.

Other edge-computing platforms build the orchestration framework with controllers similar to those used for cloud computing. For example, Cloudlet builds the controller by extending OpenStack. We believe building a specialized, scalable controller for ParaDrop can satisfy the unique requirements of edge computing.

ParaDrop API

ParaDrop implements an API for developers to create and deploy edge services. The API can be divided into two parts: the ParaDrop core interfaces implemented by the ParaDrop controller in the cloud to expose the capabilities of the management framework, and the edge node interfaces implemented by the ParaDrop daemon running on the edge nodes (ParaDrop APs) to expose the edge nodes' capabilities.

The ParaDrop core interfaces implement the resource provision, monitoring, and edge service (chute) management. With these interfaces, users can

- › register new user accounts,
- › add new APs to ParaDrop and

assign resources to be used by ParaDrop,

- › monitor APs' running status or change APs' configurations,
- › create/edit chutes, and
- › manage (configure, install, remove, start, or stop) chutes on APs.

Based on these interfaces, we developed a web frontend for users and system administrators. We also implemented an authorization framework based on OAuth 2.0 (see tools.ietf.org/html/rfc6749) so that users can authorize third-party applications to

connected to the edge service's Wi-Fi network and their signal strength, listing Bluetooth devices in the vicinity, and playing or recording audio.

System implementation

ParaDrop is an ongoing research project. We have fully implemented the two major system modules: the ParaDrop daemon on the AP and the ParaDrop controller in the cloud. These modules work together to provide the ParaDrop API. The ParaDrop daemon is implemented with the Python-based Twisted framework, and the ParaDrop controller is implemented with Node.js.

THE EDGE NODE INTERFACE PROVIDES LOCAL CONTEXT INFORMATION FOR THE EDGE SERVICES AND EXPOSES HARDWARE-SPECIFIC FUNCTIONALITY.

obtain limited access to their resources in ParaDrop. With this capability, an Internet service can dynamically deploy microservices from the cloud into edge nodes (the ParaDrop APs) close to the client and users. Developers need to register their microservices with the ParaDrop controller as chutes before integrating ParaDrop with their applications. We implemented a permission management framework so users can control which and how many resources can be used by third-party applications through the chutes.

The edge node interface provides local context information for the edge services and exposes hardware-specific functionality. Examples of edge node functions include listing devices

DEVELOPING PARADROP APPLICATIONS

Developers can build two categories of applications for ParaDrop: standalone and cloud/edge hybrid. Both categories can use ParaDrop's edge node interfaces in the chutes. ParaDrop's target applications are those requiring low latency, high bandwidth, or high privacy. Due to the relatively low-performance processor of the current ParaDrop APs, we would not recommend that developers deploy computation-intensive tasks on the APs for now. However, ParaDrop supports high-performance hardware, and we are working on the support for specialized processors (GPUs) on ParaDrop APs.

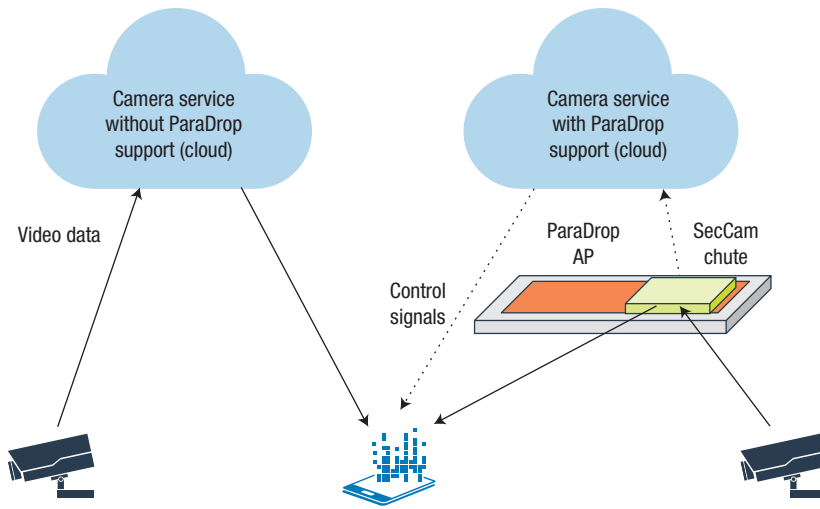


FIGURE 4. Comparison of an Internet protocol camera service without ParaDrop and with ParaDrop. Without ParaDrop, the camera needs to continuously push video data to the cloud for processing and storage. However, we can deploy a part of the cloud service on the AP with ParaDrop. If the mobile device has a direct connection with the service on the AP, it will control the camera and pull video data from it directly without relying on the service deployed in the cloud. If not, the service on the AP can still do most of the work (for example, motion detection), and it can create a peer-to-peer connection to the mobile device with the help of the simplified camera service in the cloud.

Standalone ParaDrop applications are similar to smartphone applications. Through the local dashboard in the ParaDrop AP, users can browse the available chutes and choose which to install or remove. From this perspective, we can think of a ParaDrop AP as a smart router with a controller in the cloud.

The cloud/edge hybrid ParaDrop applications are those applications having back-end servers integrated with the ParaDrop core interfaces. These applications need to be registered at www.paradrop.org to get application IDs and secret keys, and to get permissions from resource owners before the applications use ParaDrop resources. For those applications, the chutes running in the ParaDrop APs are microservices of the applications. The back-end server deployed in the cloud calls the ParaDrop core interfaces to dynamically deploy microservices (chutes) into the extreme edge (ParaDrop APs).

For reasons of space, we will not discuss the details of integrating ParaDrop core interfaces into an application, but will briefly address the chute development. More information about

ParaDrop application development, especially the cloud/edge hybrid application, is available at www.paradrop.org and in our previously published work.⁹

Through lightweight virtualization, ParaDrop provides maximum flexibility to developers. Developers can choose the programming languages and frameworks they prefer and focus on the application logic development. They can also easily port code originally written for the cloud to the ParaDrop platform with minimal effort. A chute needs to provide a Docker file to define the Linux container to execute the application logic; a configuration file to specify the environment requirements of the container, such as Wi-Fi and Bluetooth interfaces; and some application-specific files to implement application features.

IoT applications

The IoT is becoming a huge part of the networking world, but many IoT devices rely on back-end services in the cloud. Using ParaDrop, we can push that intelligence from the cloud to the edge (the APs).

SecCam. As mentioned earlier, we implemented a security camera service using a wireless video camera with a ParaDrop AP. For this service, we require network interfaces to communicate with the camera and the Internet as well as ample storage for images. Figure 4 compares the camera service deployed in the cloud and in the ParaDrop platform.

The SecCam chute is implemented with Node.js and Python. It is responsible for:

- ▶ Creating a wireless network, which provides an isolated Wi-Fi network and subnet to the security cameras.
- ▶ Capturing and processing images. A program accepts user-defined characteristics such as a threshold of motion, time of day, and rate of detection. Then it captures images from the camera, calculates differences to detect motion, and stores those images to disk.
- ▶ Providing a local server and a cloud service agent. The local server allows users to view live video, check logs, and view motion-detection images stored on the ParaDrop AP. The cloud service agent communicates with the cloud service to implement remote control and image archiving.

EnvSense. This service is a wireless environmental sensor designed as a part of the Emonix research platform.¹⁰ Because the service was fully implemented for a cloud-computing platform, we only need to migrate part of the service into the ParaDrop platform. The original service runs in the cloud to collect data from the sensors,

then processes, stores, and visualizes the data. We divide the EnvSense cloud service into multiple microservices and deploy some microservices on the ParaDrop platform. The data collection and processing microservices are deployed on ParaDrop, while the data storage and visualization microservices still run in the cloud. Due to the similarity of the ParaDrop edge nodes' runtime to the servers' runtime in the cloud, we were able to port the code written for cloud servers to ParaDrop chutes with minor effort.

ParaDrop is a flexible edge-computing platform that users and developers can use to deploy diverse services at the extreme edge of the network. ParaDrop is available as an open source project from www.paradrop.org, which also includes documents and tutorials that enable developers to begin working with ParaDrop. We have conducted tutorials and workshops with ParaDrop in multiple forums such as the US Ignite 2014 conference and the 2014 GENI Engineering Conference. Participants used ParaDrop to quickly develop services similar to SecCam, which provides preliminary evidence of its ease of use.

We are improving the platform to support more peripheral devices, such as Bluetooth and audio, and we are polishing the ParaDrop API and improving development tools to support flexible application development and deployment. In addition, we are exploring edge-computing applications for other uses based on ParaDrop. 

ACKNOWLEDGMENTS

The authors thank Mickey Barboi, Sejal Chauhan, Derek Meyer, and other students

of the University of Wisconsin–Madison for their help in the system development and deployment. This work is supported in part by the NSF through awards CNS-1555426, CNS-1525586, CNS-1405667, CNS-1345293, and CNS-1343363.

REFERENCES

1. R. Balan et al., "The Case for Cyber Foraging," *Proc. 10th ACM SIGOPS European Workshop*, 2002, pp. 87–92.
2. M. Satyanarayanan et al., "The Case for VM-based Cloudlets in Mobile Computing," *IEEE Pervasive Computing*, vol. 8, no. 4, 2009, pp. 14–23.
3. F. Bonomi et al., "Fog Computing and its Role in the Internet of Things," *Proc. 1st Edition MCC Workshop Mobile Cloud Computing*, 2012, pp. 13–16.
4. Y.C. Hu et al., *Mobile Edge Computing: A Key Technology Towards 5G*, white paper, ETSI, 2015; www.etsi.org/images/files/ETSIWhitePapers/etsi_wp11_mec_a_key_technology_towards_5g.pdf.
5. J. Gettys and K. Nichols, "Bufferbloat: Dark Buffers in the Internet," *ACM Queue*, vol. 9, no. 11, 2011, pp. 40–55.
6. D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux J.*, vol. 2014, no. 239, 2014, article no. 2.
7. S. Soltesz et al., "Container-based Operating System Virtualization: A Scalable, High-performance Alternative to Hypervisors," *Proc. 2nd ACM SIGOPS/EuroSys European Conf. Computer Systems (EuroSys 07)*, 2007, pp. 275–287.
8. W. Felter et al., "An Updated Performance Comparison of Virtual Machines and Linux Containers," *IEEE Int'l Symp. Performance Analysis of Systems and Software (ISPASS 15)*, 2015, pp. 171–172.
9. P. Liu, D. Willis, and S. Banerjee, "ParaDrop: Enabling Lightweight Multi-tenancy at the Network's Extreme Edge," *IEEE/ACM Symp. Edge Computing (SEC 16)*, 2016; ieeexplore.ieee.org/document/7774349.
10. N. Klingensmith, J. Bomber, and S. Banerjee, "Hot, Cold and In Between: Enabling Fine-grained Environmental Control in Homes for Efficiency and Comfort," *Proc. 5th Int'l Conf. Future Energy Systems (e-Energy 14)*, 2014, pp. 123–132.

ABOUT THE AUTHORS

PENG LIU is a research assistant and a PhD candidate in the Department of Computer Sciences at the University of Wisconsin–Madison. His research interests include wireless networking, video coding and streaming, and distributed systems. Liu received an MS degree in computer science from the Nanjing University of Posts and Telecommunications. Contact him at pengliu@cs.wisc.edu.

LANCE HARTUNG is an academic staff member in the Department of Computer Sciences at the University of Wisconsin–Madison. His research interests include wireless networking, distributed systems, and OSs. Hartung received an MS in computer science from the University of Wisconsin–Madison. Contact him at hartung@cs.wisc.edu.

SUMAN BANERJEE is a professor in the Department of Computer Sciences at the University of Wisconsin–Madison. He is the founding director of the WINGS laboratory, which broadly focuses on research in wireless and mobile networking systems. Banerjee received a PhD in computer science from the University of Maryland. He formerly served as the chair of ACM SIGMOBILE. Contact him at suman@cs.wisc.edu.